ResAcc

ResNet6 CIFAR-10 推論加速器

設計報告

2026 年 11 月

摘要

ResAcc 是一顆針對 CIFAR-10 影像分類網路 ResNet6 的推論加速器。輸入 32×32 彩色圖片，輸出 10 類中的一類以及該類的 confidence。網路以 8-bit 整數運算：權重 int8、activation uint8，BatchNorm 融合進卷積，每層以「整數乘法 + 右移」完成 requantize。

硬體核心是一個 16×8 的 output-stationary PE 陣列：16 列對應 16 個輸出通道，8 行對應 8 個輸出像素，部分和留在 PE 內直到整個卷積窗加完。全部權重在開機時從 DRAM 載入晶片內 SRAM 並常駐，之後每張圖只需讀入 3 KB 的影像，下一張影像的載入與目前影像的運算重疊。系統有三個時脈域：host（AXI4-Lite）、DRAM（AXI4）與運算核心，以 async FIFO 與 2-FF 同步器跨域。

739.5 µs	**64 / 64**	**63 / 64**	**159.7 k**
64 張總時間（Goal 10^6 ns 的 74%）	與整數模型 bit-exact	與浮點模型類別一致	邏輯 cells（SRAM 另計 30 KB）

項目	規格
運算精度	權重 int8（每個輸出通道一個 scale）、activation uint8、累加器 20 bit
運算陣列	128 個 8×9 乘法器（16 輸出通道 × 8 輸出像素），每 PE 兩組累加器
時脈	clk_dut 500 MHz、clk_dram 1.2 GHz、clk_host 0.77 GHz
晶片內記憶體	7 塊單埠 SRAM，共 30,128 B
每張圖	651,584 次 MAC，約 5,720 cycles（11.4 µs），PE 利用率 89%
驗證	每一層輸出與 64 張圖的最終結果都與整數參考模型逐 bit 相同；DRAM 延遲 0 / 12 cycles 與 30% 隨機空拍下結果與時間不變

目錄

1. 簡介

2. 演算法與整數量化模型

3. 運算架構（Dataflow）：架構 A、B、C 與選擇

4. 系統架構

5. 運算核心（engine）

6. 驗證

7. 實作結果

8. 結論

附錄 A　專案檔案與執行方式　　附錄 B　位址計算實例

1. 簡介

1.1 功能

Host 透過 AXI4-Lite 寫入本批的張數（1–16 張）後開始運算，之後輪詢完成旗標並讀回每張圖的結果。圖片與模型參數都存放在外部 DRAM，由加速器自行透過 AXI4 讀取。每張圖的結果為 32 bit：`[31:28]` 為類別，`[27:0]` 為 confidence（最大 logit × 2^{20}，二補數）。

1.2 規格

項目	規格
Top module	`resacc`
時脈	clk_dram 1.2 GHz（0.834 ns）、clk_host 0.77 GHz（1.3 ns）；clk_dut 自訂，本設計為 500 MHz（2.0 ns）
Reset	非同步、低電位有效
Host 介面	AXI4-Lite slave，8-bit 位址、32-bit 資料
DRAM 介面	AXI4 master，19-bit 位址、256-bit 資料，只讀
錯誤率（64 張）	< 0.37
總時間（64 張）	Baseline < 5×10^6 ns，Goal < 10^6 ns；2 張圖的時間須 < 64 張時間 / 16
模型	使用給定的訓練好模型，不可重新訓練

1.3 運算量與所需平行度

每張圖 651,584 次 MAC，64 張共 41,701,376 次。Goal 10^6 ns 在 500 MHz 下為 500,000 cycles，即使利用率 100% 也需要每 cycle 83.4 次 MAC。考慮利用率與控制開銷，本設計使用 **128 個乘法器**。

2. 演算法與整數量化模型

2.1 ResNet6

ResNet6 由一層 stem 卷積、兩個 residual block、global average pooling 與一層全連接層組成。Residual block 的主路徑是兩層 3×3 卷積，旁路（shortcut）是一層 1×1、stride 2 的卷積，兩者相加後經 ReLU 輸出。

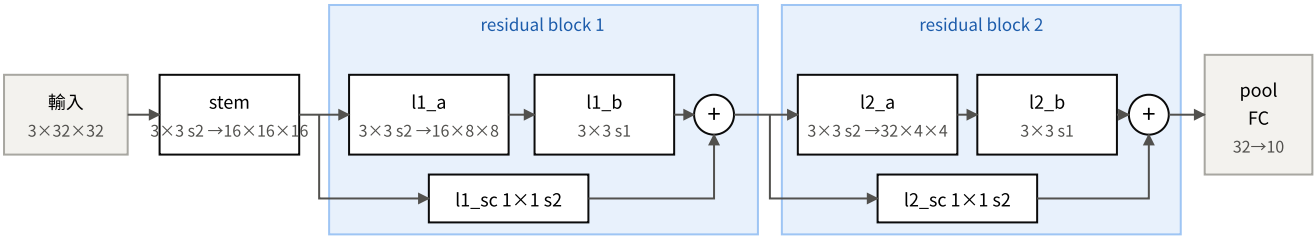


圖 2-1　ResNet6 結構（每個卷積後接 BN 與 ReLU；相加後再 ReLU）

層	輸入 → 輸出	kernel / stride	MAC / 張
stem	3×32×32 → 16×16×16	3×3 / 2	110,592
l1_a	16×16×16 → 16×8×8	3×3 / 2	147,456
l1_b	16×8×8 → 16×8×8	3×3 / 1	147,456
l1_sc	16×16×16 → 16×8×8	1×1 / 2	16,384
l2_a	16×8×8 → 32×4×4	3×3 / 2	73,728
l2_b	32×4×4 → 32×4×4	3×3 / 1	147,456
l2_sc	16×8×8 → 32×4×4	1×1 / 2	8,192
pool + FC	32×4×4 → 32 → 10	—	320
合計			**651,584**

2.2 整數量化

給定模型為 32-bit 浮點且含 BatchNorm。為了讓硬體只需整數乘加，模型以 post-training quantization 轉成整數模型，作法如下。

BatchNorm 融合

推論時 BN 是逐通道的線性運算，可併入前一層卷積：$w' = w \cdot \gamma/\sigma$，$b' = \beta - \mu \cdot \gamma/\sigma$，其中 $\sigma = \sqrt{(var + 10^{-5})}$。融合後每層只剩「卷積 + bias + ReLU」。

權重與 activation

項目	方式
實數與整數關係	$r = S \times q$，zero-point 為 0（對稱量化）
權重	int8 [−127, 127]，每個輸出通道一個 scale：$S_w[o] = \max\|w'[o]\| / 127$
activation	uint8 [0, 255]（皆在 ReLU 之後），每個 tensor 一個 scale
calibration	以第 64–191 張圖統計各 tensor 的第 99.99 百分位作為 255 對應值；第 0–63 張只用於評估
輸入圖片	$q = pixel - 128$（int8），前處理的 +0.5 偏移併入 stem 的 bias

Requantize

卷積累加值 $acc = \Sigma\, q_w \cdot q_x + b_q$ 的 scale 為 $S_w \cdot S_x$，換成下一層 scale S_y 需乘上 $M = S_w S_x / S_y$。M 以 $m / 2^s$ 近似（m 為 16-bit 無號、s 為右移量），整層運算成為：

```
一般層： y = clamp( (acc × m + 2^(s−1)) >>> s , 0, 255 )
殘差層： y = clamp( (acc_b × m_b + acc_sc × m_sc + 2^(s−1)) >>> s , 0, 255 )
```

加 2^{s-1} 為四捨五入，clamp 的下界 0 即 ReLU。m 與 bias 每個輸出通道一個，s 每層一個，殘差的兩條路徑共用 s。Global average pool 以 32 個 12-bit 總和表示（不除以 16，1/16 併入 FC 的 scale）；FC 輸出 logit 取 argmax（平手取較小 index），confidence 由 $(logit \times conf_m + 2^{conf_s-1}) >>> conf_s$ 換回實數 logit × 2^{20}。

2.3 參數範圍與準確度

層	權重形狀	s	m 範圍	$\lvert b_q \rvert$ 最大	累加需要
stem	16×3×3×3	23	11,482 – 51,844	17,472	19 bit
l1_a	16×16×3×3	23	23,548 – 64,493	30,709	18 bit
l1_sc	16×16×1×1	23	13,874 – 56,098	9,769	17 bit
l1_b	16×16×3×3	23	16,800 – 33,244	28,183	18 bit
l2_a	32×16×3×3	23	15,038 – 41,881	72,749	19 bit
l2_sc	32×16×1×1	24	20,395 – 48,057	22,494	17 bit
l2_b	32×32×3×3	24	26,767 – 41,215	36,027	19 bit
fc	10×32	conf_s 10	conf_m 33,509	6,165	20 bit

所有卷積累加值皆可放入 20-bit 有號累加器；s 只有 23 與 24 兩種值，硬體的右移因此是二選一的固定接線。

準確度	結果
整數模型與浮點模型類別一致（第 0–63 張）	63 / 64（98.4%）
整數模型與浮點模型類別一致（全部 192 張）	98.4%
confidence 平均誤差	0.08（logit 單位）
浮點模型對真實標籤的錯誤率（64 張）	0.328（< 0.37）

3. 運算架構（Dataflow）

128 個乘法器可以有不同的排列方式。排列方式（dataflow）決定每個 cycle 要從 buffer 讀多少資料、部分和存放在哪裡，以及乘法器的利用率。本章先定義三種候選架構，再比較並說明選擇。

3.1 卷積的迴圈結構

一層卷積有 Cout 個 kernel，每個 kernel 大小為 Cin×K×K。一個輸出值是一個 kernel 與輸入上對應的 Cin×K×K 區塊做內積：

```
for o in Cout:                    # 輸出通道
  for (oy, ox) in Ho×Wo:          # 輸出像素
    acc = bias[o]
    for i in Cin:                 # 輸入通道
      for ky in K: for kx in K:   # kernel 位置
        acc += W[o][i][ky][kx] × X[i][oy·s+ky−p][ox·s+kx−p]
    Y[o][oy][ox] = requant(acc)
```

可以平行展開的維度有輸出通道 o、輸出像素 (oy, ox)、輸入通道 i、kernel 位置 (ky, kx)。三種架構都使用 16×8 = 128 個乘法器，差別在於列與行分別展開哪一個維度，以及哪一種資料停留在 PE 內（stationary）。

3.2 架構 A：Output Stationary — 輸出通道 × 輸入通道

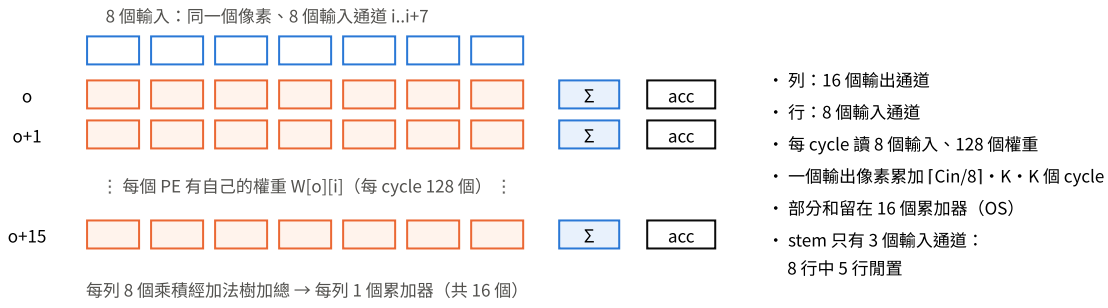


圖 3-1　架構 A：列 = 輸出通道、行 = 輸入通道，每列以加法樹加總

每個 cycle 處理一個輸出像素在一個 kernel 位置上、8 個輸入通道的乘積，一列的 8 個乘積由加法樹加總後累加到該列的累加器。所需累加器少（16 個），但每 cycle 要讀 128 個權重（1,024-bit 權重埠），而且輸入通道數不是 8 的倍數時行會閒置——stem 只有 3 個輸入通道，該層利用率只有 3/8。

3.3 架構 B：Weight Stationary — 輸出通道 × 輸入通道

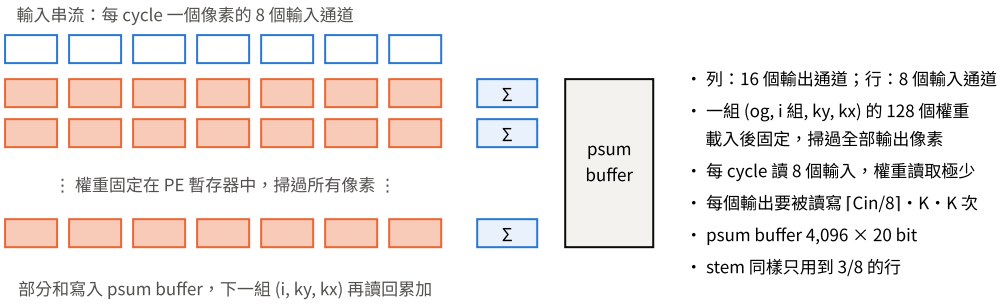


圖 3-2　架構 B：權重固定在 PE，部分和進出 psum buffer

與 A 相同的列 / 行對應，但權重載入後固定在 PE 內，輸入像素一個一個流過。權重讀取量最小，代價是部分和無法留在 PE：每處理完一組 (輸入通道組, ky, kx)，所有輸出像素的部分和都要寫回 psum buffer、下一組再讀出來累加，需要一塊 4,096×20-bit 的 psum buffer，buffer 讀寫量也大。stem 的利用率問題與 A 相同。

3.4 架構 C：Output Stationary — 輸出通道 × 輸出像素

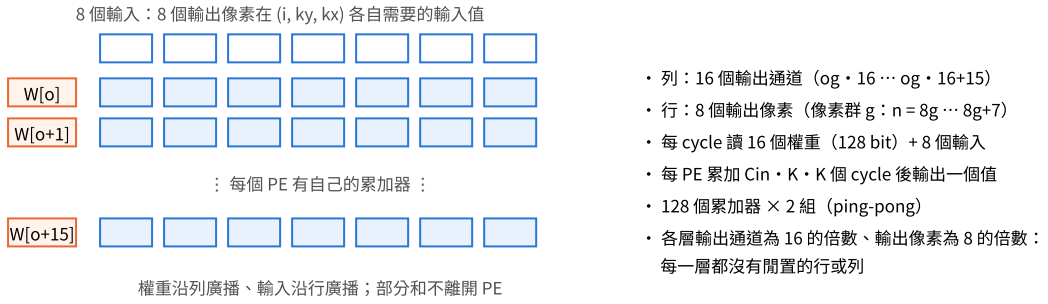


圖 3-3　架構 C：列 = 輸出通道、行 = 輸出像素

改在輸出像素方向平行：同一列的 8 個 PE 計算同一個輸出通道（共用一個權重），同一行的 16 個 PE 計算同一個輸出像素（共用一個輸入值）。每個 PE 對自己的輸出值累加整個 Cin×K×K，部分和從不離開 PE。ResNet6 每層的輸出通道數（16、32）與輸出像素數（256、64、16）都是 16 和 8 的倍數，所以每一層都能填滿陣列，包括只有 3 個輸入通道的 stem。

3.5 比較與選擇

架構	cycles / 張	利用率	平均讀取 / cycle	buffer 流量 / 張	權重埠	部分和儲存
A　OS：輸出通道 × 輸入通道	6,532	78%	106 B	685 KB	1,024 b	16 個累加器 + 加法樹
B　WS：輸出通道 × 輸入通道	6,532	78%	46 B	535 KB	1,024 b	psum buffer 81,920 b
C　OS：輸出通道 × 輸出像素	**5,120**	**99%**	**24 B**	**128 KB**	**128 b**	128 × 2 個累加器

理想排程下的數字（不含控制開銷），由 `97_ARCH/dataflow.py` 計算。

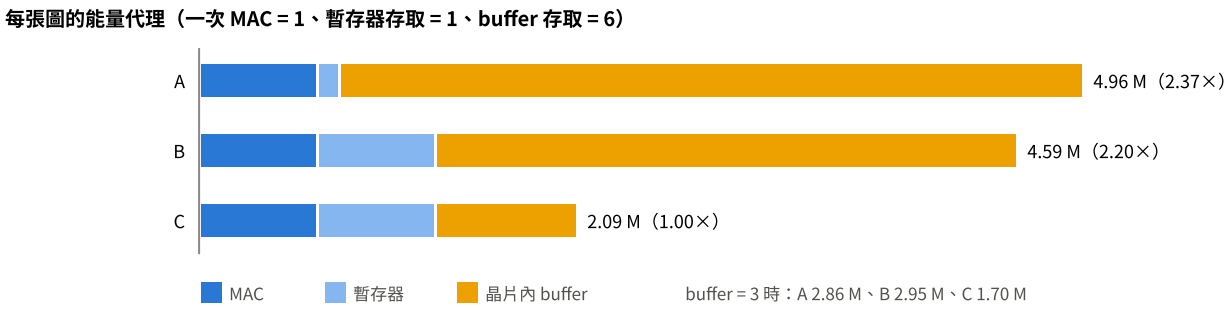


圖 3-4　三種架構的能量代理（相對耗能取自 Eyeriss / MIT DNN 加速器教材）

選擇架構 C，理由如下：

- **時間**：利用率 99%，每張圖 5,120 cycles，比 A、B 少 22%；在 500 MHz 下 64 張的理想時間為 655 µs，留有足夠餘裕給控制開銷。
- **能量**：每 cycle 只讀 16 個權重與 8 個輸入（同一個 SRAM word），buffer 流量只有 A 的 19%，能量代理約為 A、B 的一半；buffer 相對耗能取 3 或 6，排名都不變。
- **面積**：累加器較多（128×2×20 bit），但不需要 B 的 81,920-bit psum buffer，也不需要 A、B 的 1,024-bit 權重讀取埠與 16 棵加法樹。

DRAM 頻寬方面，C 每張圖運算 5,120 cycles，而讀入一張圖片只要 96 個 DRAM beat、全部權重 624 個 beat，因此權重可以在開機時一次載入並常駐晶片，之後只需讀入影像，且影像載入完全可以和運算重疊。

4. 系統架構

4.1 頂層方塊圖

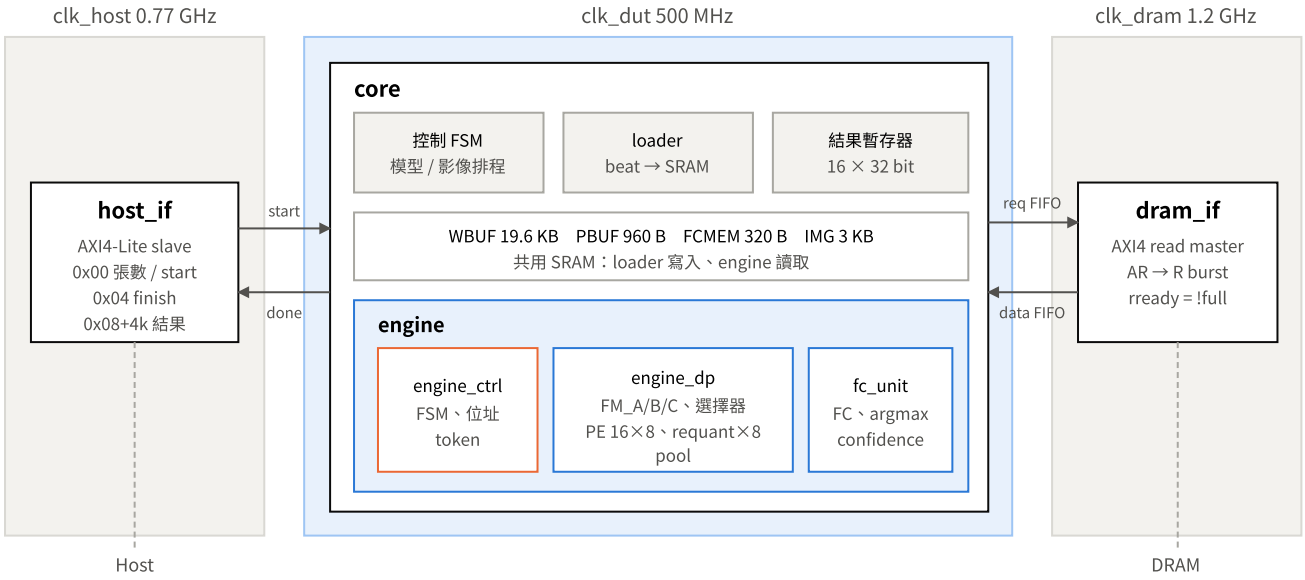


圖 4-1　ResAcc 頂層方塊圖

模組	時脈域	功能
`host_if`	clk_host	AXI4-Lite slave、暫存器、start 與 finish 的跨域處理
`dram_if`	clk_dram	AXI4 讀取狀態機；不寫 DRAM
`async_fifo` ×2	dut ↔ dram	讀取請求（深度 4 × 27 bit）、讀回資料（深度 16 × 256 bit）
`core`	clk_dut	流程控制、loader、共用 SRAM、運算核心 engine、結果暫存器

4.2 時脈域與跨域（CDC）

訊號	方向	做法
讀取請求	dut → dram	`async_fifo`：讀寫指標轉 Gray code 後以 2-FF 同步到對方時脈域；指標比位址多 1 bit 以區分滿與空
讀回資料	dram → dut	`async_fifo`，寫入端以 `rready = !full` 對 DRAM 背壓
start	host → dut	host 每寫一次 0x00 翻轉一次 toggle；dut 端 2-FF 同步後偵測翻轉。張數在翻轉前已寫入且保持不變，翻轉被偵測到後才取樣
done	dut → host	準位訊號經 2-FF 同步，host 端偵測上升緣後設定 finish；host 寫 0x00 時立即清除 finish，不會讀到上一批的旗標
結果暫存器	dut → host	不經同步（quasi-static）：done 拉起前結果已固定，host 只在看到 finish 後讀取；合成時設為 false path

每個時脈域各有一個 `reset_sync`，reset 非同步拉低、同步放開。

4.3 Host 介面（`host_if`）

位址	存取	內容
0x00	W	`[3:0]` = 本批張數 − 1；寫入即開始一批
0x04	R	`[0]` = finish
0x08 + 4k	R	第 k 張結果：`[31:28]` 類別、`[27:0]` confidence

AW 與 W 可分開或同時到達，兩者都收到後才執行寫入，下一個 cycle 回應 B；AR 收到後下一個 cycle 回應 R。一次只處理一筆交易。

4.4 DRAM 介面與資料配置

`dram_if` 為三狀態的讀取狀態機：從請求 FIFO 取出 {位址, 長度} → 送出 AR → 接收 R beat 寫入資料 FIFO 直到 `rlast`。一次只有一筆讀取在進行，burst 一律不跨 4 KB 邊界。DRAM 每個 beat 為 32 bytes，資料配置如下：

byte 位址	內容	大小	讀取方式
0x00000 + 0x1000・n	第 n 張圖片（CHW，一個 beat = 一列 32 個像素）	96 beats	每張圖 1 個 burst
0x10000	WSEC：所有卷積權重	614 beats	開機時 5 個 burst（128/128/128/128/102）
0x15000	PSEC：shift、bias、m、FC 權重與 bias	43 beats	開機時 1 個 burst

模型檔（`model.dat`）中的資料**依硬體使用順序排列**，硬體只需依序寫入 SRAM，不需要任何位址轉換：

- **WSEC**：以 128-bit word（16 個 int8 權重）為單位，word 內 byte r 是第 r 個輸出通道的權重，正好對應 PE 陣列的第 r 列。順序為 `for layer: for og: for i: for ky: for kx`，共 1,227 個 word；每個 DRAM beat 放兩個 word。
- **PSEC**：beat 0 為 header（7 層的 s、conf_m、conf_s）；beat 1–30 為 10 個參數區塊，每個區塊 3 beats（bias 0–7、bias 8–15、m 0–15），一個區塊對應一層的一組 16 個輸出通道；beat 31–40 為 FC 權重（每 beat 一個類別的 32 個權重）；beat 41–42 為 FC bias。

4.5 Loader

`loader` 依目前的載入工作，把資料 FIFO 的 beat 寫入對應的記憶體：

工作	beat	寫到	cycle / beat
模型（開機一次）	WSEC 0–613	WBUF，每 beat 拆成兩個 128-bit word	2
	PSEC 0	header 暫存器（7 個 s、conf_m、conf_s）	1
	PSEC 1–30	PBUF[0..29]	1
	PSEC 31–40	FCMEM[0..9]	1
	PSEC 41–42	FC bias 暫存器	1
影像（每張）	b = c×32 + y	IMG word c×16 + y/2 的低半（y 偶）或高半（y 奇），以 byte enable 寫入	1

4.6 Core 控制流程

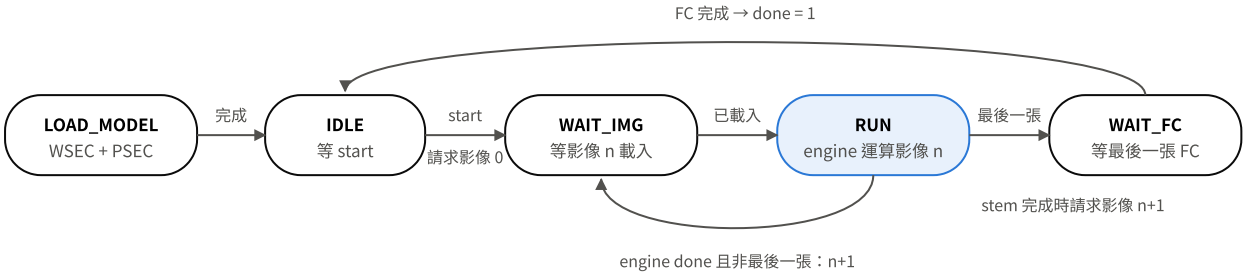


圖 4-2　core 的控制狀態機

重置後先載入模型（約 1,300 cycles，只做一次）。收到 start 後請求第 0 張影像；每張影像運算時，engine 一做完 stem 層（IMG 不再被讀取）就回報 `stem_done`，core 立刻請求下一張影像，使影像載入與運算重疊。FC 與下一張影像的前幾層同時進行；整批最後一張的 FC 完成後設定 done。



圖 4-3　影像載入、運算與 FC 的重疊

5. 運算核心（engine）

5.1 結構

engine 分為 control path 與 datapath。`engine_ctrl` 只包含狀態機、迴圈計數器與位址計算，決定每個 cycle 要做什麼；`engine_dp` 包含 feature map SRAM、運算元選擇、PE 陣列、requant 與 pool，只依照收到的控制資訊運作。兩者之間以 **token** 溝通：`engine_ctrl` 每個 cycle 送出一個 token，描述這一拍資料的所有控制資訊（第幾個 bank、是否為該批的第一拍 / 最後一拍、每一行要取哪個 byte、是否為 padding、排出時寫到哪裡等），token 隨資料逐級前進，每一級使用與自己資料同一拍的控制位元。

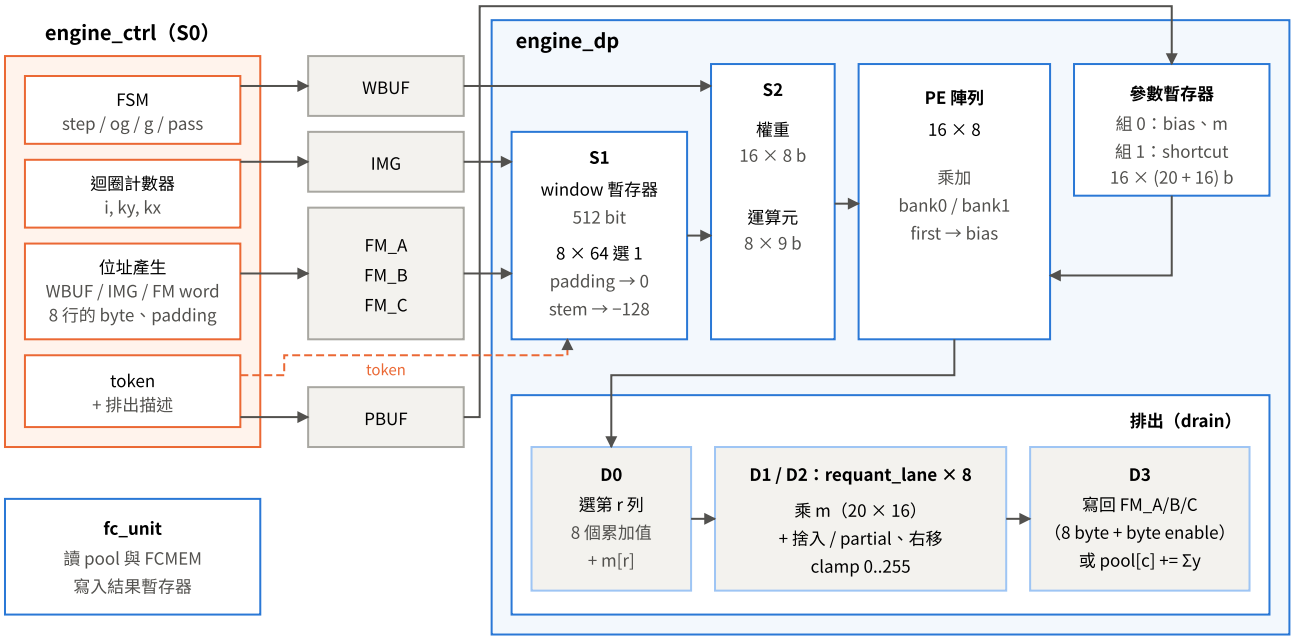


圖 5-1　engine 方塊圖（虛線為 token；FM_A/B/C 位於 engine_dp 內，WBUF / IMG / PBUF 位於 core）

5.2 晶片內記憶體

所有 SRAM 皆為單埠、讀取延遲 1 cycle、支援 byte enable、內容不需 reset。

名稱	深度 × 寬度	容量	內容
WBUF	1,227 × 128 b	19,632 B	全部卷積權重；一個 word = 一組 16 個輸出通道在某個 (i, ky, kx) 的權重
PBUF	30 × 256 b	960 B	10 個參數區塊（bias、m）
FCMEM	10 × 256 b	320 B	FC 權重，一個 word = 一個類別
IMG	48 × 512 b	3,072 B	輸入影像
FM_A	64 × 512 b	4,096 B	stem 輸出
FM_B	16 × 512 b	1,024 B	l1_a 輸出；之後存 l2_a 輸出
FM_C	16 × 512 b	1,024 B	residual block 1 輸出
合計		**30,128 B**	

Feature map 存放格式

IMG 與 FM_A/B/C 的一個 word 為 64 bytes。形狀 C×H×W 的 tensor，plane = H×W：

```
plane ≥ 64：  rows_per_word = 64 / W
              word = c × (plane / 64) + y / rows_per_word ,  byte = (y mod rows_per_word) × W +
x
plane < 64：  ch_per_word = 64 / plane
              word = c / ch_per_word ,  byte = (c mod ch_per_word) × plane + y × W + x
```

tensor	形狀	位置	每 word	word 數
影像	3×32×32	IMG	2 列	48
stem 輸出	16×16×16	FM_A	4 列	64
l1_a 輸出	16×8×8	FM_B	1 個通道	16
block 1 輸出	16×8×8	FM_C	1 個通道	16
l2_a 輸出	32×4×4	FM_B	4 個通道	8
block 2 輸出	32×4×4	不儲存，直接累加到 pool		

此格式保證兩個性質：(1) 一個像素群在某個 (i, ky) 需要的 8 個輸入一定在同一個 word 內，每個 (i, ky) 只讀一次 SRAM；(2) 一個像素群的 8 個輸出是同一個 word 內連續且對齊 8 的 8 個 byte，寫回只需一個 word 加 byte enable。

5.3 工作劃分：步驟、批、pass

一張影像分成 5 個步驟。每個步驟由若干**批**組成：一批 = 16 個輸出通道（第 og 組）× 8 個輸出像素（第 g 個像素群），共 128 個輸出值，正好佔滿 PE 陣列。殘差步驟的每一批包含兩個 pass：先算 shortcut（1×1），再算主路徑（3×3）。一個 pass 依序走過 (i, ky, kx)，每 cycle 一組。

步驟	層	讀	寫	像素群	批數	每批 cycle	排出
0	stem	IMG	FM_A	半列	32	27	一般
1	l1_a	FM_A	FM_B	一列	8	144	一般
2	l1_sc + l1_b	FM_A、FM_B	FM_C	一列	8	16 + 144	殘差
3	l2_a	FM_C	FM_B	兩列	2 × 2 og	144	一般
4	l2_sc + l2_b	FM_C、FM_B	pool	兩列	2 × 2 og	16 + 288	殘差 + pool

每個步驟的讀與寫都落在不同的 SRAM，單埠 SRAM 不會發生讀寫衝突。

5.4 Control path（`engine_ctrl`）

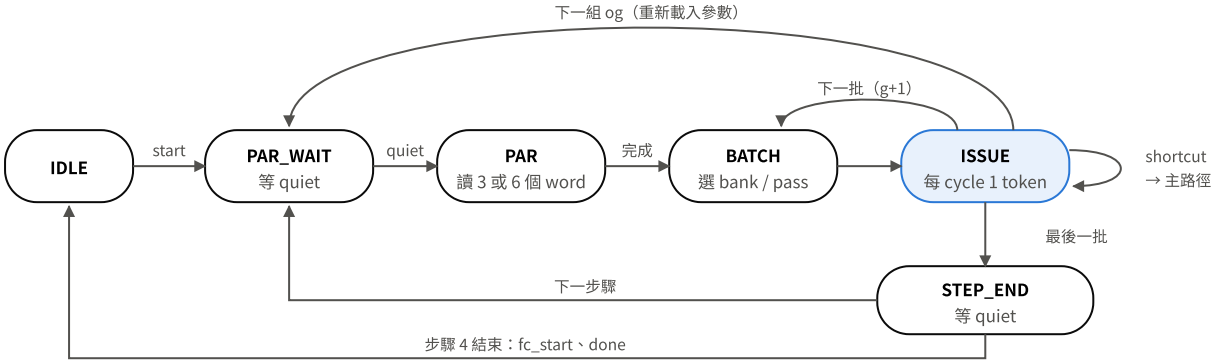


圖 5-2　engine_ctrl 的狀態機

狀態機：每換一個 (步驟, og) 先在 PAR 從 PBUF 讀出參數區塊——一般層 3 個 word 寫入參數暫存器組 0，殘差層再讀 3 個 word（shortcut 區塊）寫入組 1。參數只在管線與排出都清空（**quiet**）時更換；殘差步驟的每一批開始前也等 quiet，因為兩個累加器 bank 都會被這一批使用。步驟 3 結束時若上一張影像的 FC 仍在讀 pool，會等它結束再清除 pool。

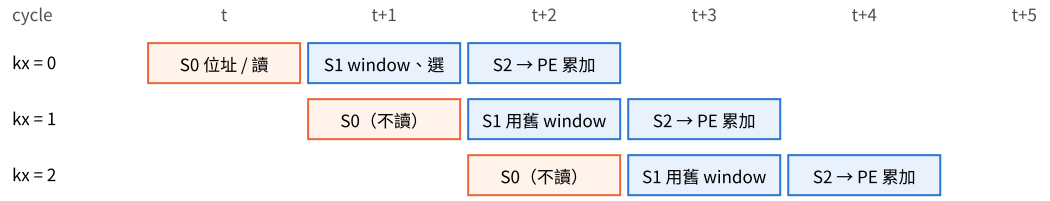
迴圈與權重位址：ISSUE 狀態每 cycle 推進 (i, ky, kx)（1×1 層只推進 i），權重位址每 cycle 加 1，因為 WBUF 中權重正是依這個順序排列；每個 pass 開始時載入該層、該 og 的起始位址。

輸入位址：第 p 行計算輸出像素 n = 8g + p。由 (oy, ox) 與 (ky, kx) 算出輸入座標 `iy = oy·stride + ky − pad`、`ix = ox·stride + kx − pad`，再依 5.2 的格式得到 word 位址（以第 0 行、夾在 [0, H−1] 的 iy 計算）與每一行在 word 內的 byte 位置；座標超出邊界者標為 padding。8 行的位置計算由 generate 展開成 8 份相同的組合邏輯。SRAM 只在 kx = 0 時讀取。

Token 內容：

欄位	用途	使用階段
`valid, first, last, final`	有效拍、pass 第一拍（載入 bias）、pass 最後一拍、此 pass 結束一批（啟動排出）	S1–PE
`bank, bset`	累加器 bank、使用的參數組	PE
`win_ld, src, stem`	更新 window、資料來源 SRAM、運算元減 128	S1
`sel[8×6], vld[8]`	每一行在 window 中的 byte 位置、padding 旗標	S1
`dmode, ddst, dword, dbyte, dpacked, dwstep, dog, ds24`	排出描述：一般 / 殘差 / pool、目的 SRAM、起始 word 與 byte、寫回位址步距、shift 為 23 或 24	排出

5.5 運算管線（`engine_dp`）



每個 cycle 送出一個 token；一個 (i, ky) 的 3 個 kx 共用 kx = 0 時讀出的 window。
PE 的累加結果在 S2 結束時寫入累加器。

圖 5-3　運算管線時序（3×3 層的一個 (i, ky)）

階段	動作
S0（engine_ctrl）	送出 WBUF 位址（每 cycle）與 IMG / FM 位址（kx = 0 時），產生 token
S1	SRAM 資料到達。依 `src` 選擇 IMG / FM_A / FM_B / FM_C；`win_ld` 時新 word 寫入 512-bit window 暫存器，否則沿用舊值。8 個 64 選 1 選擇器依 `sel` 取出 8 個 byte，轉成 9-bit 有號運算元：padding 為 0、stem 為 pixel − 128、其他層為零擴展的 activation
S2	16 個權重（WBUF 讀出的 128 bit）與 8 個運算元進入暫存器，送進 PE 陣列；同時依 `bset` 選出 16 個 bias

5.6 PE 與 PE 陣列

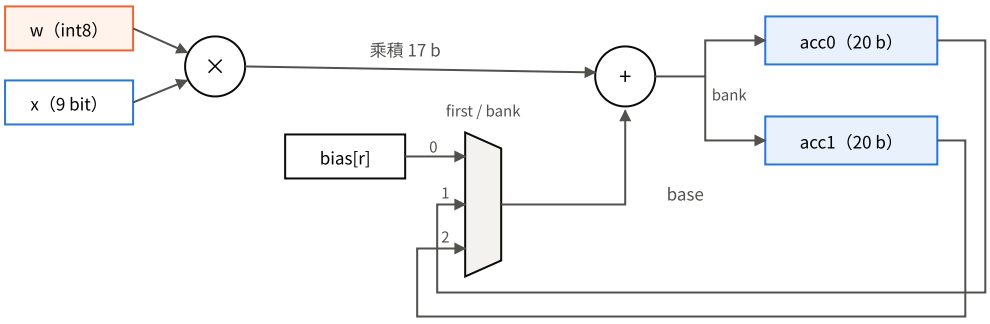


圖 5-4　單一 PE：先依 first / bank 選出加數的基底（bias 或該 bank 的累加值），再與乘積相加，結果只寫回 token 指定的 bank

每個 PE 有一個 8×9 有號乘法器、一個 20-bit 加法器與兩組 20-bit 累加器。批的第一拍以 bias 為初值，因此不需要另外清 0。一般層兩個 bank 輪流使用，一個在累加下一批時，另一個正在被排出；殘差層 shortcut 使用 bank1、主路徑使用 bank0。

PE 陣列以 generate 展開 128 個 PE（第 r 列第 p 行）。第 r 列接收權重 byte r 與 bias[r]，第 p 行接收運算元 p，所有 PE 共用 en、bank、first 控制。陣列提供一個讀出埠：輸入列號 r，同時輸出該列 8 個 PE 的 bank0 與 bank1 累加值，供排出使用。

5.7 排出與 requant

一批的最後一個 token（`last && final`）到達 PE 時啟動排出序列器，它把 128 個累加值逐列取出：一般層每列 1 cycle、16 cycle 完成；殘差層每列 2 cycle、32 cycle 完成。排出描述在啟動時由 token 鎖存，因此下一批可以立即開始累加（一般層使用另一個 bank）。

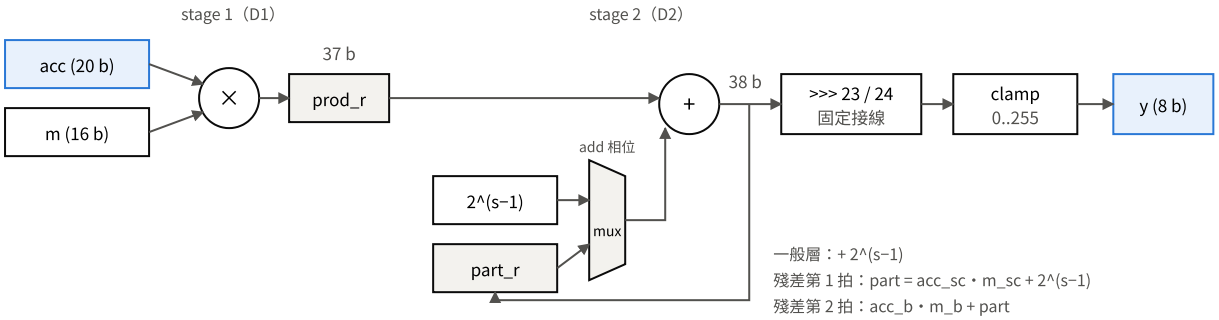


圖 5-5　requant_lane（共 8 條，每條一個乘法器、一個加法器）

階段	一般層（每列 1 cycle）	殘差層（每列 2 cycle）
D0	讀出該批 bank 的第 r 列與 $m_0[r]$	第 1 拍：bank1（shortcut）與 $m_1[r]$；第 2 拍：bank0（主路徑）與 $m_0[r]$
D1	8 條通道各自 acc × m	
D2	$+ 2^{s-1}$，右移，clamp → y	第 1 拍：結果（含捨入常數）存入 part；第 2 拍：+ part，右移，clamp → y
D3	寫回或加入 pool（5.8）	

5.8 寫回與 pool

D3 時 8 個輸出 y[0..7] 屬於同一個輸出通道 c = og×16 + r、連續 8 個像素。寫回時把 8 個 byte 複製 8 份成 512 bit，以 one-hot 的 8-byte byte enable 寫入目標 word 的對應位置；word 與 byte 位址由排出描述與列號 r 算出。步驟 4（block 2）不寫回，而是把 8 個 y 相加後累加到 `pool[c]`（32 個 12-bit 暫存器），完成 global average pool 的加總。

5.9 FC 單元（`fc_unit`）

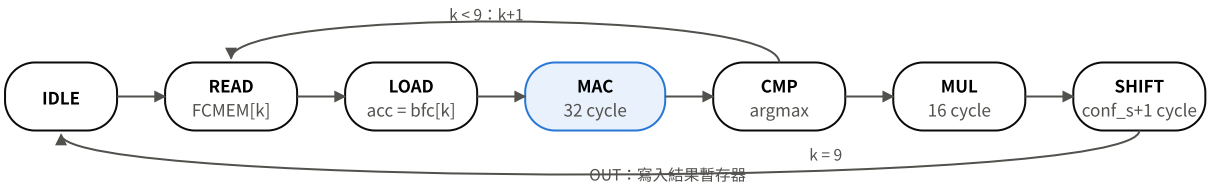


圖 5-6　fc_unit 的狀態機

10 個類別依序處理：讀出該類別的 32 個權重，以 FC bias 為初值，用一個 13×8 乘法器做 32 cycle 的乘加（32-bit 累加器）。每個類別結束後比較大小，只有嚴格大於目前最大值才更新，因此平手時保留較小的類別編號。最後的 confidence 乘法以 16 cycle 的 shift-and-add 完成（每 cycle 處理 conf_m 的一個 bit），接著加上捨入常數並每 cycle 算術右移 1 bit，共 conf_s 次，取低 28 bit 與類別組成 32-bit 結果寫入結果暫存器。整個 FC 約 380 cycles，與下一張影像的運算同時進行。

6. 驗證

6.1 參考模型

模型	內容	確認
`float_ref.py`	浮點參考模型（numpy，不依賴 PyTorch）	與 PyTorch 輸出一致；BN 融合前後一致
`quant.py`	整數模型，硬體的 golden model；輸出每一層的累加值、activation、pool、logit	與浮點模型類別一致 63/64
`hw_model.py`	依硬體規格運作的模型：從 `model.dat` 讀參數，依 5.2 的格式與 5.3 的批次順序計算	與 `quant.py` 192/192 bit-exact

6.2 測試環境



圖 6-1　系統層級測試環境

層級	對象	比對
單元	`sram_sp`、`async_fifo`（6 種讀寫時脈與速率組合，各 20,000 筆隨機資料）	讀取延遲、byte enable；資料順序與內容、不溢位
周邊	全部周邊 RTL + engine 替身	模型載入後 WBUF / PBUF / FCMEM / header 與 hw_model 逐 word 比對；每張影像的 IMG 比對；host 讀寫路徑
逐層	完整系統	每個步驟結束時 FM_A / FM_B / FM_C / pool 換算回 tensor，與 GOLDEN 逐 byte 比對
系統	完整系統，64 張（4 批 × 16 張）	每張圖的類別與 confidence 與 `golden_results.txt` 比對

6.3 結果

測試	結果
單元測試	全部通過
周邊測試（64 張）	64 / 64 通過
逐層比對（64 張 × 5 個步驟）	320 / 320 一致
系統（READ_LATENCY 12）	64 / 64 bit-exact；與浮點模型類別一致 63 / 64
系統（READ_LATENCY 0）	64 / 64 bit-exact

系統（READ_LATENCY 12、30% 隨機空拍）	64 / 64 bit-exact

7. 實作結果

7.1 效能

項目	結果
64 張總時間（READ_LATENCY 12）	739,541.5 ns（Goal 的 74.0%）
64 張總時間（READ_LATENCY 0 / 30% 空拍）	739,518.1 ns / 739,541.5 ns
2 張總時間	26,527.9 ns（< 739,542 / 16 = 46,221 ns）
每批 16 張	184,228.2 ns
開機載入模型	約 2.6 µs（只有一次）

每張影像各步驟的實測 cycle 數（穩態，第 2 張影像）：

步驟	實測 cycles	純運算 cycles	其餘時間
stem	923	864	參數載入、步驟結束等待排出
l1_a	1,187	1,152	同上
l1 殘差	1,593	1,280	每批 32 cycle 排出（陣列暫停）
l2_a	633	576	兩組 og 各載入一次參數
l2 殘差	1,384	1,216	每批 32 cycle 排出
每張影像	**5,720**	**5,088**	**PE 利用率 89%**

FC 約 380 cycles，與下一張影像重疊；下一張影像的載入（96 beats）在 stem 結束後開始，於該影像其餘 4 個步驟期間完成，不在關鍵路徑上，因此 DRAM 延遲與空拍不影響總時間。

7.2 面積

SRAM 共 30,128 B（約 24 萬 bit）；以 16 nm 小型 SRAM macro 每 bit 0.15–0.25 µm² 估計約 37,000–62,000 µm²。邏輯以 Yosys 合成至 generic gates（SRAM 視為黑盒子）：

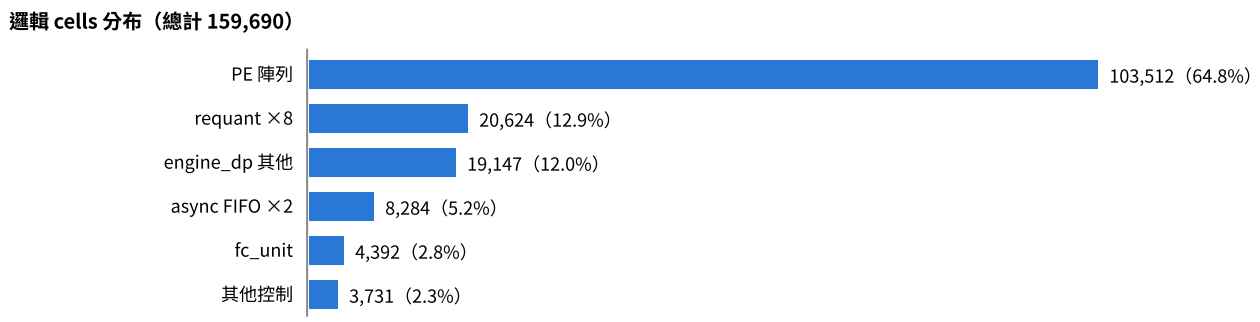


圖 7-1　邏輯面積分布

模組	cells	主要內容
pe_array	103,512	128 個 PE（每個 731 cells：8×9 乘法器、20-bit 加法器、2 × 20-bit 累加器）+ 列讀出選擇器
requant_lane × 8	20,624	每條一個 20×16 乘法器、38-bit 加法器、clamp
engine_dp 其他	19,147	512-bit window、8 個 64 選 1 選擇器、參數暫存器、pool、管線暫存器
async_fifo（資料 / 請求）	8,042 / 242	資料 FIFO 為 16 × 256-bit 暫存器陣列
fc_unit	4,392	13×8 乘法器、32-bit 累加器、48-bit shift-add
engine_ctrl	1,248	狀態機、計數器、位址產生
host_if / loader / core / dram_if / 同步器	948 / 796 / 655 / 42 / 42	
resacc 合計	**159,690**	**其中 flip-flop 14,201 個**

7.3 邏輯深度

模組	最長路徑（gate 層數）	路徑
pe	44	8×9 乘法 + 20-bit 加法
engine_ctrl	45	輸入位址與 byte 位置計算
requant_lane / engine_dp	77	38-bit 加法 + clamp（排出 D2）
fc_unit	109	13×8 乘法 + 32-bit 累加

Yosys 以 ripple-carry 展開加法器，數字只適合相對比較。若要在實際製程達到 500 MHz，requant 與 FC 的乘法路徑可再各切一級管線；在 token 架構下只需讓 token 多經過一級暫存器。

8. 結論

ResAcc 以 8-bit 整數量化的 ResNet6 為對象，採用「輸出通道 × 輸出像素」的 output-stationary dataflow，以 128 個乘法器達到 89% 的 PE 利用率。權重常駐晶片、影像載入與運算重疊，使 DRAM 延遲完全不影響效能；64 張影像總時間 739.5 µs，為 Goal 的 74%。RTL 的每一層輸出與 64 張影像的最終結果都與整數參考模型逐 bit 相同。

可以進一步改進的方向：將參數暫存器做成雙組，換輸出通道組時不必等排出完成；殘差層兩個 bank 輪替以消除排出時陣列暫停；利用 stride 規律縮小 8 個 64 選 1 選擇器；以開源製程 library 進行時序分析與實體設計。

附錄 A　專案檔案與執行方式

位置	內容
`98_ALGORITHM/`	`resnet6.pth`、`resnet6*.ipynb`（原始模型與 notebook）；`common.py`、`float_ref.py`、`quant.py`、`sweep.py`、`hw_model.py`（參考模型）；`gen_model_dat.py`（產生 model.dat 與 golden_results.txt）；`quant_params.npz`
`97_ARCH/`	`arch_model.py`、`dataflow.py`、`compare.py`、`schedule_C.py`：第 1、3 章的數字
`00_TESTBED/`	`TESTBED.sv`、`host_model.sv`、`stub/`、`unit/`、`periph/`、`layer/`；`PATTERN/`：192 張影像、`categories.csv`、`confidences.csv`、`model.dat`、`golden_results.txt`、`GOLDEN/`
`10_AXI_DRAM/`	`axi_dram.sv`
`01_RTL/`	RTL、`rtl.f`、`engine.f`、`run_sys.sh`、`run_periph.sh`
`02_SYN/`	`run_yosys.sh`：面積與邏輯深度

```
# 需要 Python 3 + numpy、Icarus Verilog（-g2012）；Yosys 為選用
cd 98_ALGORITHM
python3 quant.py              # 整數模型、GOLDEN/
python3 gen_model_dat.py      # model.dat、golden_results.txt
cd ../00_TESTBED/unit && sh run_unit.sh
cd ../../01_RTL
sh run_periph.sh 64           # [張數] [READ_LATENCY] [RAND_GAP%]
sh run_sys.sh 64 12 0         # 完整系統：逐層 + 最終結果比對
cd ../02_SYN && sh run_yosys.sh
```

附錄 B　位址計算實例

l1_a，og = 0，g = 3（輸出第 3 列），i = 5，ky = 0：

項目	計算	結果
權重位址（kx = 0）	27 + 0 + 5×9 + 0×3 + 0	WBUF word 72
輸入列	iy = 3×2 + 0 − 1	5
輸入 word	FM_A：每 word 4 列、每通道 4 word → 5×4 + 5/4	word 21
該列在 word 內的起點	(5 mod 4) × 16	byte 16

kx	ix（p = 0..7）	選出的 byte
0	−1, 1, 3, ⋯, 13	padding、17、19、21、23、25、27、29
1	0, 2, ⋯, 14	16、18、20、22、24、26、28、30
2	1, 3, ⋯, 15	17、19、21、23、25、27、29、31

排出第 r 列時，輸出通道 c = r、像素 (3, 0..7)，寫入 FM_B 的 word c、byte 24–31。